

# CityWise Jaipur – Next-Gen Civic Issue Reporting & Resolution Platform

CityWise is a connected full-stack civic resolution platform built specifically for **Nagar Nigam Greater & Heritage Jaipur**. It powers transparent civic complaint lifecycle management through automated GeoJSON point-in-polygon ward routing, department authority assignment, real-time Socket.IO notifications, live camera evidence capture, side-by-side resolution proof inspection, and mandatory citizen verification.

## ⚡ Supabase PostgreSQL Database Setup

CityWise natively supports **Supabase PostgreSQL** alongside its built-in SQLite engine.

### 1. Create a Supabase Project

1. Sign up / Log in to [Supabase Dashboard](#).
2. Click **New Project** and name it `citywise-jaipur`.

### 2. Apply Schema Migration

1. Open your project's **SQL Editor** in Supabase Dashboard.
2. Open `data/supabase_schema.sql` from this repository.
3. Paste the entire SQL script into the Supabase SQL Editor and click **Run**.

### 3. Set Environment Variables

Copy `.env.example` to `.env` in the `backend/` directory:

```
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your-supabase-service-role-key
SUPABASE_ANON_KEY=your-supabase-anon-key
```

### 4. Seed Supabase Database

Run the automated Supabase seeder to populate Jaipur Parshads and initial civic complaints:

```
cd backend
npm run seed:supabase
```

## Jaipur Municipal Region & Official Data Integration

CityWise integrates official ward parshad representative dataset from **Nagar Nigam Greater Jaipur (पं. दीनदयाल उपाध्याय भवन, लालकोठी, टोंक रोड जयपुर)** across all 5 municipal zones:

- **Vidhyadhar Nagar Zone (Wards 1–42)** (e.g. Ward 1 Smt Dhapa Devi, Ward 15 Dr Meenakshi Sharma)
- **Jhotwara Zone (Wards 43–64)** (e.g. Ward 43 Smt Archana Sharma, Ward 60 Smt Sheel Dhabai)
- **Sanganer Zone (Wards 65–103)** (e.g. Ward 70 Shri Ramavtar Gupta, Ward 87 Mayor Smt Saumya Gurjar)
- **Bagru Zone (Wards 104–124)** (e.g. Ward 104 Shri Arun Sharma, Ward 120 Shri Chhoturam Meena)
- **Malviya Nagar Zone (Wards 125–150)** (e.g. Ward 125 Shri Ramprasad Sharma, Ward 142 Shri Himanshu Jain)

## Complete Resolution Workflow

```
REPORT
↓
GEOLOCATE (Interactive Leaflet Map)
↓
IDENTIFY WARD (Point-in-Polygon GeoJSON Engine)
↓
ASSIGN AUTHORITY (Department Lookup: PWD Roads, PHED Water, Sanitation)
↓
ACT (Authority Status: IN_PROGRESS)
↓
SUBMIT RESOLUTION (Authority Uploads Live Camera Photo/Video Proof)
↓
CITIZEN VERIFY (Citizen Approves or Reopens with Remarks)
↓
CLOSE (Verified Resolution)
```

## Tech Stack

- **Frontend:** React 18, Vite, Tailwind CSS, React Router v6, React Leaflet (OpenStreetMap), Recharts, Socket.IO Client, Axios, Lucide Icons, MediaDevices WebRTC Camera Capture.
- **Backend:** Node.js, Express.js, Supabase JS Client ( `@supabase/supabase-js` ), SQLite, Socket.IO Server, JWT, bcryptjs, Multer, Turf.js ( `@turf/boolean-point-in-polygon` ).
- **Data & Testing:** Supabase SQL Schema ( `data/supabase_schema.sql` ), GeoJSON Ward Polygons ( `data/wards.geojson` ), Native Node.js Test Runner ( `node --test` ).

## Project Structure

```
CityWise/
├── frontend/
│   ├── src/
│   │   ├── components/    # CameraCapture, Navbar, Footer, MapPicker, StatusBadge, IssueTimeline
│   │   ├── context/       # AuthContext, SocketContext
│   │   ├── pages/         # Landing, About, Community, PublicMap, Login, Register, Dashboards,
│   │   └── services/      # Axios API client
│   ├── index.html
│   ├── vite.config.js
│   └── package.json
├── backend/
│   ├── database/          # citywise.sqlite
│   ├── uploads/           # Uploaded photos & videos
│   ├── src/
│   │   ├── config/        # DB schema & Supabase client (supabaseDb.js)
│   │   ├── middleware/    # Auth JWT, RBAC, Multer upload
│   │   ├── routes/        # Auth, Wards, Authorities, Categories, Issues, Incidents, Notifications
│   │   ├── services/      # Ward GeoJSON detection, Authority routing, Status Machine, Duplicate
│   │   └── seed/          # Jaipur Parshad demo seeder (seedData.js & seedSupabase.js)
│   ├── server.js
│   └── package.json
├── data/
│   ├── supabase_schema.sql # Official Supabase PostgreSQL schema migration
│   └── wards.geojson       # Official Jaipur municipal ward polygons & Parshad details
├── tests/
│   └── citywise.test.js    # Integration test suite
├── docker-compose.yml
├── README.md
├── .env.example
└── .gitignore
```

## Demo Login Accounts

| ROLE                   | EMAIL                   | PASSWORD    | DETAILS                         |
|------------------------|-------------------------|-------------|---------------------------------|
| Citizen                | citizen@citywise.org    | password123 | Rajesh Sharma (Jaipur Resident) |
| PWD Roads Officer      | roads@citywise.org      | password123 | Engineer Vikram Singh           |
| PHED Water Officer     | water@citywise.org      | password123 | Officer Sunita Meena            |
| Nagar Nigam Sanitation | sanitation@citywise.org | password123 | Inspector Mohan Lal             |
| System Admin           | admin@citywise.org      | admin123    | Nagar Nigam Master Control      |

## Running Locally (Windows Instructions)

### 1. Backend Server Setup

```
cd backend
npm install
npm run seed           # Seeds local SQLite database
npm run seed:supabase  # (Optional) Seeds Supabase PostgreSQL if SUPABASE_URL is configured
npm start              # Runs Express & Socket.IO server on http://localhost:5000
```

### 2. Frontend Application Setup (Open a second terminal)

```
cd frontend
npm install
npm run dev           # Runs Vite dev server on http://localhost:3000
```

## Docker Deployment

To build and run the entire application using Docker:

```
docker compose up --build
```

Access Frontend at `http://localhost:3000` and Backend API at `http://localhost:5000`.

## Testing

---

Run the automated integration test suite:

```
node --test tests/citywise.test.js
```

Tests cover:

- Database schema & Supabase compatibility
- GeoJSON Point-in-polygon ward detection
- Ward-to-Authority department lookup
- Status machine transition validation
- Duplicate issue spatial grouping
- Live Weather API integration & Predictive Risk correlation